



建议

学习数学切忌直接背结论公式，应该学会自己推导证明公式，因为比赛中如果是推导公式的题，那么这个公式肯定是你没有学过的。如果比赛中会用到某个你学过的公式，那么你不会数学推导肯定是想不到的，所以学习数学没有捷径可以走。



数论基础

简单来说，信息学竞赛中所有和数学有点的知识都可以称为数论。这几年信息学竞赛题目难度逐渐由代码量偏向思维量，其中思维量中很大一部分是在考察数学能力。

就和思维没有边界一样，数论也没有边界，如果有，那就是无限大。今天主要讲解数论的基础，也就是是一些常见的，经常用到的数学知识。

主要包括三点：

- (1) 素数相关
- (2) 逆元相关
- (3) 排列组合



素数

什么是素数：

定义：在大于1的自然数中，除1和他本身之外没有其他因子的自然数被称为素数。





素数

如何证明一个数是否是素数？



素数

可以用定义来直接证明，如果一个数 n 是素数，那么除了1和 n 之外，他没有其他因子，也就是说对于 $2 \sim n-1$ ，他们都不能被 n 整除。

时间复杂度： $O(n)$



素数

如何证明一个数很大的数是否是素数？
 $n < 10^{10}$



素数

我们思考，对于一个数 n 如果他是质数，那么 $2—n-1$ 都不能被他整除，那么如果他不是质数呢？那么 n 就一定可以写成两个数之积，而这两个数都是 n 的因子。

即： $n=p*q$

如果 p 是一个比较小的因子，那么 q 就是一个比较大的因子， p, q 之间一定有一个数 $\leq \sqrt{n}$ 。

我们思考，如果在 $2—\sqrt{n}$ 中不存 n 的因子，那么是否可以证明 n 就是素数？

完全是可以的。我们可以用反证法：

证明：如果 n 有一个因子 p 大于 $\sqrt{n}$ ，那么 n 就能整除 p ，那么 n/p 就一定小于 $\sqrt{n}$ ，但是在 $2—\sqrt{n}$ 中并不存在 n 的因子，所以 n 一定是质数。



素数

如果 n 更大呢？ $n \leq 100$ 位



素数

对于特别大的数，我们会发现前面这种枚举到 $\sqrt{n}$ 的办法也还是会超时了，这就要用到我们的一些高级数学知识和数学定理。

其中有一种快速证明一个数是否是素数的方法叫：

Miller-Rabbin算法

后面再讲。



素数

一般情况下，判断是否是素数只是其中的一个小部分，所以一般数据不会特别大，但是他可能不是询问某一个数是否是素数，而是问1— n 中有多少个素数，或者1— n 中有哪些素数？这又该如何计算呢？



素数

我们会发现目前我们会的证明素数只有枚举，素数不容易证明是因为他要枚举完他的所有因子才能判断，在数学中，如果我们没有办法直接证明的时候，我们可以去证明他的反面。

既然素数我们不好直接求解，那我们如果求出所有的合数，那么剩下的数就肯定是素数。

那我们要如何才能判断一个数是否是合数呢？

实际上判断是一个很麻烦的过程，我们完全可以根据合数的定义来构造：一个数的倍数(>1)一定是合数。

素数

比如2的倍数：2, 6, 8, 10...

3的倍数：6, 9, 12

4的倍数：8, 12, 16...

我们通过一个数的倍数所生成的数一定是合数，并且只要我们一次枚举这个数，那么合数是不会有遗漏的。

所以我们可以先枚举2—n，再枚举倍数。这样就可以把所有的合数标记出来，剩下未标记的就是素数。

比如10以内的素数：

2的倍数：4 6 8 10

3的倍数：6 9

4的倍数：8

5的倍数：10

6, 7, 8, 9, 10的倍数：无

素数

但是这样做最大的问题在于时间复杂度太大了，首先要枚举 n 个数，然后再枚举倍数，相当于双重循环。所以我们要想办法优化。

我们很容易想到第一种优化方法：

如果用 $2*5$ 筛选出了10，那么我们就完全没必要再用 $5*2$ 来筛选10了，也就是说对于一个合数 $a*b$ ，当我们用 a 筛选了之后，就没有必要再用 b 来筛选了。

怎么才能避免这种重复呢？我们只需要保证 $a \leq b$ 就可以了，所以我们在枚举的时候还是只需要枚举 $2 \sim \sqrt{n}$ 就可以了。我们以 $1 \sim 16$ 为例： $\sqrt{16}=4$

2的倍数：4 6 8 10 12 14 16

3的倍数：6 9 12 15

4的倍数：4 8 12 16

素数

实际上我们还可以继续优化，我们每次只需要让小的那个因子来筛选这个数就可以了，比如合数6，他可以写成 2×3 ，再用2的倍数筛选的时候就筛选了6，那么当我们再用3来筛选的时候就没有必要再去筛选6了。所以我们可以让倍数 $\geq$ 该数，也就是枚举3的倍数直接从3开始，枚举4的倍数直接从4开始，这样就避免了重复。

而且，我们会发现4的倍数都被2的倍数筛选过了，又重复筛选了一次，很浪费时间，我们既然是筛选合数，那么如果这个合数已经被筛选过，那么这个合数的倍数是不是肯定也会被筛选？

比如2筛选了4 6 8 10，那么可以想象，肯定也会筛选4, 6, 8, 10的倍数，所以我们只需要枚举素数，不需要枚举合数。

素数

这样一来，一种高效的筛选素数的方法就出来了。

下面是模拟过程，以1—30为例

2的倍数：4 6 8 10 12 14 16 18 20 22 24 26 28 30

3的倍数：9 12 15 18 21 24 27 30

4的倍数：已经被标记了，就不在筛选了

5的倍数：25 30

6的倍数： $6*6 > 30$ 不再筛选

这种通过筛选合数的方法来求解区间的素数的方法是一种常见的求解区间素数的方法，效率也比较高，差不多是 $n \log n$

素数

```
int t=sqrt(n)
for(int i=2;i<=t;i++)//枚举因子
{
    if(!book[i])//没有被标记说明是质数
    {
        for(int j=i;i*j<=n;j++)//枚举倍数
            book[i*j]=1;//标记合数
    }
    //如果被标记那么说明是合数，不用合数筛选
}
```


素数

我们再仔细观察一下筛选1—30之间的数的方法，看看是否有数被重复筛选，实际上是有的。

其中12, 18, 24被筛选了2次，30被筛选了3次。理论上来说这种筛选是不应该存在重复筛选的数的，那么为什么会出现呢？我们把这些数分解一下：

$$12=2*2*3$$

$$18=2*3*3$$

$$24=2*2*2*3$$

$$30=2*3*5$$

原来原因在于这些数的因子有很多个，而每次拆分都满足我们筛素数的条件。比如30，用2筛选时： $2*15$, $15>2$ ，用3拆分时。 $3*10$, $10>3$. 用5的倍数拆分时， $5*6$, $6>5$

素数

也就是说这种拆分方法虽然高效，但是仍然还是存在重复个数，而且重复被筛选的次数还不少。

我们假设一个数可以写成 $2^a * 3^b * 5^c \dots$ ，那么他就可能被筛选他的质因子的个数次。着也就导致了这种筛选方法的时间复杂度趋近于 $O(n)$ 但是却高于 $O(n)$ 。

那么我们有没有办法让我们的筛选更加完美呢？也就是说保证我们的每一个数都只会被筛选一次。

素数

一个合数，他可能由多个质因子组成，这是没有办法避免的，但是当我们把一个合数拆分成多个质数相乘的形式，肯定有一个质数是最小的，那么对于任意的一个数，我们能不能只用他的最小质因子来筛选这个合数呢？

比如 $30=2*3*5$ ，他的最小质因子为2，那么我们只在 $2*15$ 的时候把30筛选了，后面 $3*10$ ， $5*6$ 就不再筛选了。

比如在用3做筛选的时候，

$3*3=9$ (筛选)

$3*4=12$ (但是12的最小质因子是2，不筛选)

$3*5=15$ (筛选)

$3*6=18$ (18的最小质因子是2，不筛选)

...

素数

但是这在代码中应该如何实现呢？我们怎么才知道一个数的最小质因子是多少？对于每一个合数再去分解一次吗？其实并不是。

我们可以换一种筛选方法，前一种筛选方法是枚举质数，再枚举倍数，但是倍数可能是一个合数，所以不好处理。我们可以倒过来，先枚举倍数，再枚举质数。

我们可以先枚举倍数 i ，再枚举已经得到的质数 $\text{num}[j]$ ，去筛选 $i * \text{num}[j]$ 这个质数，因为 num 中存储的质数是有序的，如果 $i \% \text{num}[j] == 0$ 表示 i 的最小质因子就是 $\text{num}[j]$ ，那么就没有必要再往后面枚举 j 了，因为对于后面的 $i * \text{num}[k]$ ，他的最小质因子一定不是 $\text{num}[k]$ 而是 $\text{num}[j]$ 。

素数

对于数30

倍数2(未标记)	筛选 $2*2$
倍数3(未标记)	筛选 $3*2$ $3*3$
倍数4(被标记)	筛选 $4*2$ ($4\%2==0$)
倍数5(未标记)	筛选 $5*2$ $5*3$ $5*5$
倍数6(被标记)	筛选 $6*2$ ($6\%2==0$)
倍数7(未标记)	筛选 $7*2$ $7*3$
倍数8(被标记)	筛选 $8*2$ ($8\%2==0$)
倍数9(被标记)	筛选 $9*2$ $9*3$ ($9\%3==0$)
倍数10	筛选 $10*2$
倍数11	筛选 $11*2$
倍数12	筛选 $12*2$
倍数13	筛选 $13*2$
倍数14	筛选 $14*2$
倍数15	筛选 $15*2$

素数

```
for(int i=2;i<=n;i++)//枚举倍数
{
    if(book[i]==0)//如果倍数是素数
    {
        num[total]=i;
        total++;
    }
    for(int j=1;j<=total&& i*num[j]<=n;j++)//枚举找到的质数
    {
        book[i*num[j]]=1;
        if(i%num[j]==0)//说明i的最小质因子是num[j], 后面的不用筛了
            break;
    }
}
```

虽然这两种筛选素数的方法的代码是比较类似，但是他的双重循环的含义完全不一样，两者是反过来的，一定要理解清楚，不能死记硬背，否则后面肯定会混淆。

素数

我们先已经学会了如何快速求解区间1— n 以内的素数，但是有的时候他不是一个完全的1— n 的区间，而是区间 $[n, m]$ ，但是不同的地方在于 $n, m \leq 10^{12}$ ，但是 $m - n \leq 10^6$ ，也就是说区间长度不大，但是区间所处的位置很大，遇到这种题我们又该如何来求解呢？

我们会发现，如果我们对于每一个数去判断他是否是素数，时间复杂度为 $\sqrt{10^{12}} * 10^6$ ，肯定会超时。如果我们用前面筛选素数的方法，就算是严格的 $O(n)$ ，时间复杂度也是 $O(10^{12})$ ，效率也是特别低的。

素数

其实我们可以思考一下，虽然 $n, m \leq 10^{12}$ ，但是如果 n, m 是合数，那么他必有一个质因子 $\leq \sqrt{m}$ ，所以我们在枚举质因子的时候还是只需要筛选到 $\sqrt{m}$ ，但是我们以前筛选的时候都是用质因子*从2开始的倍数来筛选，如果还这样筛选肯定时间空间上都不能满足要求，既然我们只需要求出区间 $[n, m]$ 的质数，那么我们枚举倍数就完全没必要再从2开始了，而是直接从 n/i 开始，枚举到 $i * \text{num}[j] \leq m$ 的时候就结束，这样时间复杂度总体还是控制在 $\max(\sqrt{m}, (m-n) \log(m-n))$ 之间，还是可以接受的

思路

- 1: 线性筛选出 $\sqrt{m}$ 中的所有素数
- 2: 用筛选出来的素数再去筛选 $[n, m]$ 之间的素数。

```
for(int i=1;i<=cnt;i++)//枚举sqrt(m)内的素数
{
    for(long long j=max((n-1)/num[i]+1,(long long)2);num[i]*j<=m;j++)
    {
        flag[num[i]*j-n]=1;//都在区间[n,m]内,可以都左移n防止空间开爆
    }
}
```



无平方因子数

给出正整数 n 和 m ，区间 $[n, m]$ 内的“无平方因子”的数有多少个？整数 p 无平方因子当且仅当不存在 $k > 1$ ，使得 p 是 k^2 的倍数。 $1 \leq n \leq m \leq 1e12, m - n \leq 1e7$ 。



无平方因子数

其实该题和前面筛素数的做法是一样的，首先，如果一个数是平方因子数，那么他肯定有一个因子 $\leq \sqrt{n}$ ，所以我们还是只需要枚举 $\sqrt{n}$ 以内的素数就可以了。

同样，我们只需要枚举素数的平方，在枚举倍数，因为非素数的平方都可以有素数的平方*倍数得到。

同样，该题区间所处的位置很大，但是区间长度很小，所以还是要注意*倍数的时候，直接保证在区间内，所以该题实际上就是一道筛选素数的裸题。

质因子

我们都知道，任何一个合数都可以写成多个质数的乘积，并且这些质数是固定的。也就是唯一分解定理：

唯一分解定理：每一个大于1的正整数均可分解为有限个素数的积，如果不计素因数在乘积中的次序，则分解方式是唯一的。将n的素因数分解中相同的素因子收集到一起，可只每个大于1的正整数n可唯一地写成 $n = p_1^{a_1} * p_2^{a_2} * p_3^{a_3} \dots p_k^{a_k}$ ，其中， $p_1, p_2, p_3, \dots, p_k$ ，是互不相同的素数，而 $a_1, a_2, a_3, \dots, a_k$ 是正整数，上面的分解式称为n的标准分解。

质因子

唯一分解定理的正确性也很容易证明，假设一个数的分解方式不止一种，那么 $p = a_1 * a_2 * \dots * a_i = b_1 * b_2 * \dots * b_i$. 那么等式两边同时除以 a_1 (或者任何一个质因子)，那么左边仍然是一个整数，但是右边，因为所有因子都是质数，那么右边的每一个数都和 a_1 互质(除非相同)，那么等式右边除以 a_1 之后，就不再是一个整数了，所以该分解方式是错误的。



唯一分解

给定一个数 n ，对这个数进行分解，并从小到大输出这些质因子。 $n \leq 10^{12}$;



唯一分解

首先，很容易想到分解质因数的方法，先一直除以2，除到不能整除了，再一直除以3，再一直除以5. 既然是分解质因子，那么首先肯定要筛选出质数。同样，该题是 10^{12} ，如果我们筛选出所有的质数，无论时间还是空间都不能接受，还是和前面考虑的方法一样，如果一个数有质因子 $>\sqrt{n}$ ，那么这样的质因子有且只有一个，那么我们完全只需要枚举 $1\text{---}\sqrt{n}$ ，剩下就用 $n/\text{前面质因子之积}$ 得到，而不是一直枚举到 n 。



因子数

给定一个数 n ，求出这个数的所有因子
比如30，他的因子就有1, 2, 3, 5, 6, 10, 15, 30
 $n \leq 10^{12}$.



因子数

一种朴素做法就是枚举 $1 \sim n$ ，通过判断是否可以整除来判断该数是否是 n 的因子，但是这样枚举时间复杂度很大。如果我们只枚举到 $1 \sim \sqrt{n}$ 呢，完全是没有问题的，因为对于枚举的每一个数 a ，那么肯定还有一个因子 (n/a) 与之对应

因子数

```
void fenjie(int x)
{
    int k=2;
    int total=0;
    while(k*k<=x)
    {
        if(x%k==0)
        {
            p[++total]=k;
            while(x%k==0)//对因子重数的其他处理
            {
                sum[total]++;
                x/=k;
            }
        }
        k++;
    }
    if(x>1) p[++total]=x;sum[total]++;
    //再对分解最后的那个质数进行处理
}
```



因子数

求出 $[1, n]$ 之间每个数的因子的集合。
 $n < 10^6$



因子数

如果我们还是按照前面的做法分别求出每个数的因子有哪些，那么时间复杂度就是 $n * \sqrt{n}$ ，所以我们需要找到一个更快的办法。

既然是区间统计，那么我们就没有必要一个一个算，一个数 a 的倍数是 b ，那么 a 肯定就是 b 的因子，所以我们可以把求用试除法求因子数转换成用倍数来求解。

因子数

```
vector<int> v[1000100]; // 可变数组
int n;
int main()
{
    scanf("%d",&n);
    for(int i=1;i<=n;i++)
    {
        for(int j=1;j<=n/i;j++)
            v[i*j].push_back(i);
    }
    for(int i=1;i<=n;i++)
    {
        for(int j=0;j<v[i].size();j++)
        {
            printf("%d ",v[i][j]);
        }
        printf("\n");
    }
}
```



因子数

如果给定的这个数就是唯一分解形式呢？

比如已知一个数 $a_1^{b_1} * a_2^{b_2} * \dots * a_n^{b_n}$, 问这个数的因子数有多少个？





因子数

该题如果元素特别大，那么我们就没有办法把他计算出来，那么我们应该如何求解他的因子呢？实际上我们都知道，对于每一个数的因子实际上都可以有一或者多个质数的乘积表示。所以我们完全可以从唯一分解中任意选择几个质因子来乘起来就是该数的一个因子，那么这样的因子一共有多少个呢？



因子数

对于任意的一个因子 $a_i^{b_i}$, 那么只考虑该因子选择的数量, 那么就是可以选1个, 2个, 最多选择 b_i 个, 那么就有 b_i+1 种可能, 那么一共就有 $(b_1+1)*(b_2+1)*(b_3+1)*\dots*(b_n+1)$ 种可能。同样因子数也是这么多种。

那么一个数的因子数之和是多少?

因子数

我们还是就根据唯一分解的性质来思考，我们假设把唯一分解看成两部分 $a_1^{b_1} * f(x)$ ，那么因子数之和为 $f(x) + a_1 * f(x) + a_1^2 * f(x) + \dots + a_1^{b_1} * f(x) = (1 + a_1 + a_1^2 + \dots + a_1^{b_1}) * f(x)$ ，我们再把 $f(x)$ 分开考虑，实际上最终得到的答案就是

$$(1 + a_1 + a_1^2 + \dots + a_1^{b_1}) * (1 + a_2 + a_2^2 + \dots + a_2^{b_2}) * \dots * (1 + a_n + a_n^2 + \dots + a_n^{b_n})$$

实际上就转换成了等比数列求解。

$$a_1 + a_1 * q + a_1 * q^2 + \dots + a_1 * q^{(n-1)} = a_1 * (1 - q^n) / (1 - q)$$

$$\text{证明 } q * S_n - S_n = (q^n - 1) / (q - 1)$$



阶乘分解

给定整数 N ($N \leq 10^6$), 试把阶乘 $N!$ 分解质因数,
按照算术基本定理的形式输出分解结果中的 p_i 和 c_i 。



阶乘分解

直接算肯定是算不出来答案的，我们可以考虑对于 $[1, n]$ 分别分解质因数，然后相同的项再合并，最后就可以得到 p_i 和 c_i ，但是分解质因数的时间复杂度为 $\sqrt{n}$ ，总体时间复杂度为 $O(n * \sqrt{n})$ 。

阶乘分解

遇到这种区间求解的题我们一般都是整体考虑，即直接考虑倍数，一个数 a 的倍数是 b ，那么 a 就是 b 的因子，首先，分解得到的一定是质数，所以我们筛选出 $[1, n]$ 之间的素数，在考虑次方数，在 $[1, n]$ 中，对于数字 p ，出现了 n/p 次，那么 n/p 是否是 $n!$ 中 p 的次数呢，我们来尝试一下。

$$4! = 24 = 2^3 * 3$$

2的次数是3，并不是2，为什么会出现这种问题呢？因为 $[1, n]$ 中还有 2^2 这个因子，对于大一点的数，还有 $2^3 \cdots 2^i$ 的因子，所以一个数 p 的次数应该是

$$n/p + n/(p^2) + n/(p^3) + \cdots n/(p^k) \quad k = \log(n)$$



因子数

求 n 以内与 n 互质的数的个数



因子数

我们还是很容易想到一种很朴素的做法，枚举1— $n-1$ ，判断该数与 n 的最大公约数是否是1，判断最大公约数是可以利用欧几里得算法，也就是辗转相除法。那么我们是否可以只判断到 $\sqrt{n}$ 呢，实际上并不行，因为这里只是问是否有相同因子，而不是一个数是否是另一个数的因子，所以我们必须要枚举完，总的时间复杂度为 $O(n) * \log(n)$ 。

那么，如果 n 特别大呢？ $n \leq 10^{12}$

因子数

我们思考，如果 n 是一个质数，那么 n 以内与 n 互质的数那么肯定就是 $(n-1)$ 个。

那么如果 $n=p*q$ (p, q 都是质数)，那么 n 以内与 n 不互质的数有多少个呢？很明显只有

$p, 2p, 3p \cdots (n/q)p, q, 2q, (n/p-1)*q$ ，因为 p, q 是互质的，所以里面的数都不重复，所以这样的数有 $p*q-p-q+1=(p-1)*(q-1)$ 个。

如果 $n=p*q$ (p, q 不一定是质数，但是 p, q 互质)，在 $1 \cdots (p*q-1)$ 中，只有既与 p 互质 ($f(p)$) 的数，又与 q 互质 ($f(q)$) 的数，才会与 $p*q$ 互质，这样的数一共有 $f(p)*f(q)$ 个。即 $f(p*q)=f(p)*f(q)$ (p, q 互质)

(见欧拉函数的积性证明)

因子数

那么如果一个数 $n = a^p$ (a 是质数), 那么 n 以内与 n 互质的数有多少个?

因为 a 是质数, 那么 n 的质因子只有 a , 那么只有 a 的倍数才和 n 不互质, 那么在 $[1, a]$ 中与 a 互质的数有 $(a-1)$ 个, $[a+1, 2a]$ 中与 a 互质的数有 $(a-1)$ 个, $[a^p - a + 1, a^p]$ 中与 a 互质的数有 $(a-1)$ 个, 那么总的与 a^p 互质的数一共有 $a^p * (1 - 1/a)$ 个。也就是 $a^p - a^{p-1}$ 。

那么对于任意一个数都可以写成 $a_1^{b_1} * a_2^{b_2} * a_3^{b_3} \dots$ 那么每个因子的阶乘都是互质的, 所以可以用积性函数的乘法性质得到

$$f(x) = x * (1 - 1/a_1) * (1 - 1/a_2) * \dots * (1 - 1/a_i);$$

也就是欧拉函数



欧拉函数

1— n 之间与 n 互质的数之和是多少？



欧拉函数

我们仔细观察 n 以内的质数有哪些，以及之和是多少就会发现一个规律，即 $f(n) * n / 2$.

怎么证明呢？很简单

如果一个数 a 与 n 互质，那么 $(n-a)$ 也与 n 互质

很容易证明， $(n-a) = k * x$ $n = k * y$

那么 $a = k(y-x)$ ，如果 $(n-a)$ 与 n 不互质，那么 a 与 n 也不互质，所以证明不成立。

欧拉函数

前面我们学习的欧拉函数的相关知识，其中都要用到求 n 以内与 n 互质的数的个数，对于一个数 n ，求解其欧拉函数的时间复杂度为 $O(\sqrt{n})$ ，但是如果我们需要求解 $1--n$ 之间所有数的欧拉函数呢？如果在一个一个去计算太麻烦了，有没有什么更快的办法呢？

可以直接用公式 $f(a*b)=f(a)*f(b)$ (如果 a, b 互质)

欧拉函数

如果 n 为质数, $f(n)=n-1$

如果 $n\%i==0$ (i 为质数) $f(n*i)=f(n)*i$

如果 $n\%i\neq 0$ (i 为质数) $f(n*i)=f(n)*(i-1)$

欧拉函数

证明：已知 $f(x) = x * (1 - 1/a_1) * (1 - 1/a_2) * \dots * (1 - 1/a_i)$

如果 $x \% i == 0$, 那么 i 就是 x 的质因子, $x * i$ 的质因子和 x 的质因子是相同的, 那么对于 $x * i$

$$f(x * i) = (x * i) * (1 - 1/a_1) * (1 - 1/a_2) * \dots * (1 - 1/a_i)$$

$$\text{即 } f(x * i) = f(x) * i$$

如果 $x \% i \neq 0$

那么 $x * i$ 比 x 多一个质因子 i

$$\text{所以 } f(x * i) = (x * i) * (1 - 1/a_1) * (1 - 1/a_2) * \dots * (1 - 1/a_i) * (1 - 1/i)$$

$$f(x * i) = f(x) * i * (1 - 1/i) = f(x) * (i - 1)$$

欧拉函数

```
for(int i=2;i<=n;i++)
{
    if(!book[i])
    {
        prime[++cnt]=i;
        phi[i]=i-1; // 质数的欧拉函数
    }
    for(int j=1;j<=cnt&& i*prime[j]<=n;j++)
    {
        book[i*prime[j]]=1;
        if(i%prime[j]==0) // 如果可以整除
        {
            phi[i*prime[j]]=phi[i]*prime[j];
            break;
        }
        phi[i*prime[j]]=phi[i]*(prime[j]-1); // 如果不可以整除
    }
}
```

poj 3090

在一个平面直角坐标系的以 $(0,0)$ 为左下角， (N,N) 为右上角的矩形中，除了 $(0,0)$ 之外，每个坐标上都插着一个钉子，求在 origin 向四周看去，能够看到多少个钉子，一个钉子能被看到，当且仅当连接它和 origin 的线段上没有其他钉子。 $N \leq 1000$.

poj 3090

自习观察就会发现，除了 $(0, 0)$, $(0, 1)$, $(1, 0)$ 之外，其他点 (x, y) 被看到当且仅当 $\gcd(x, y) = 1$ ，即横纵坐标互质，也及时对于一个数 x ，所有与 x 互质的数有多少个，所以很明显就是欧拉函数。

所以该题就是欧拉函数的线性筛之和+3就是答案



素数的性质

前面我们学习了如何筛选素数以及如何分解素数。素数实际上还有很多重要的性质，也有很多重要的公式和素数有关。



威尔逊定理

如果 p 是一个素数，那么 $((p-1)!+1)\%p=0$

反过来，证明仍然成立。也就是说这是一个充分必要定理。

威尔逊定理

充分性证明(正着证): 略

必要性证明(反着证):

如果 p 不是素数, 那么 $p=a*b$

如果 $a \neq b$

那么 $(p-1)! \% a = 0$ $(p-1)! \% b == 0$

则: $(p-1)! \% (a*b) = 0$

$(p-1)! \% p = 0$ 与 $((p-1)! + 1) \% p = 0$ 矛盾

如果 $a == b$

同理。所以 p 肯定是素数



费马小定理

如果 p 是一个质数，则 $a^{(p-1)\%p}=1$ (a 与 p 互质)
正着一定成立，反着不一定成立



费马小定理

证明：

$1a, 2a, 3a \cdots (p-1)a$ 分别模 p 的余数是 $[1, p-1]$ 的某种排列，也就是说余数都不相同。

这个很好证明，如果 $s*a \% p = t*a \% p$

那么 $(s-t)*a \% p = 0$ ，但是 p 的质数，那么所以小于 p 的数都与 p 互质，所以 $(s-t)$ 与 p 互质， a 与 p 互质，所以证明不成立。

那么我们把这这些数都乘起来

$$a * 2a * 3a * \cdots (p-1)a = (1 * 2 * 3 \cdots (p-1)) \% p$$

$$a^{(p-1)} * (p-1)! = (p-1)! \% p$$

$(p-1)!$ 肯定与 p 互质，所以可约去，所以

$$a^{(p-1)} = 1 \% p \quad \text{即：} a^{(p-1)} \% p = 1$$

费马小定理

费马小定理的逆定理不一定成立。

如果满足 $a^{(p-1)} \% p = 1$, 则 p 一定是素数。

比如 341, 561, 1005..., 这样的数被称为反素数, 而且反素数的个数为无限多个, 所以不能用该定理直接判断一个数是否是素数。就算不停的更换 a , 也有一些合数永远满足对于所有数 a , 都有 $a^{(p-1)} \% p = 1$

欧拉定理

如果 a, n 互质，那么 $a^{f(n)} \% n = 1$

$f(n)$ 表示欧拉函数，即 n 以内与 n 互质的数的个数

反素数

题目描述

[展开](#)

对于任何正整数 x ，其约数的个数记作 $g(x)$ 。例如 $g(1)=1$ 、 $g(6)=4$ 。

如果某个正整数 x 满足： $g(x) > g(i)$ $0 < i < x$ ，则称 x 为反质数。例如，整数1，2，4，6等都是反质数。

现在给定一个数 N ，你能求出不超过 N 的最大的反质数么？

输入格式

一个数 N ($1 \leq N \leq 2,000,000,000$)。

输出格式

不超过 N 的最大的反质数。

输入输出样例

输入 #1

[复制](#)

输出 #1

[复制](#)

1000

840

反素数

这题看上去是一道数论题，实际上确定一道带技巧性的搜索题，必须要掌握里面的规律，否则爆搜肯定T

结论一：1—N中的反素数就是1—N中因子最多的数中最小的那一个

结论二：根据数据范围，因子数不会超过10个，因为 $2*3*5\cdots 31 > 2e9$

结论三：相同质因子最多30个，因为 $2^{31} > 2e9$

结论四(重要结论)：反素数的质因子一定是连续若干个最小的质数，并且指数单调递减。设连续质数分别为 $x_1, x_2, x_3\cdots x_n$ ，不存在 $x_i \ x_{i+2}$ 这种情况

反素数

证明：

如果存在反素数 $x_1 * x_2 * \dots * x_i * x_{i+2}$ ，那么必有 $x_1 * x_2 * \dots * x_i * x_{i+1}$ 值比前面一个数小并且因子数一样。

同样，如果连续质因子的质数不递减，也可以用前面的替换方法来得到一个比当前数小但是因子数量一样多的数。

所以我们直接跑深搜，只需要筛选出前10个质数，搜索的时候指数递减搜索就可以了。